Ch15 小考——運算子的多載 Operator Overloading

Problem 1

考試日期： 2026-05-18

目錄

Problem 1

題目敘述

定義一個 Point（點）的類別擁有 x 及 y 座標資料成員，再透過 Point 類別宣告 `p1(10, 20)`、`p2(15, 23)`、`p3(0, 0)` 三個物件。最後多載 + 一元運算子（前置運算）及 + 二元運算子的功能。運算子功能分別說明如下：

1. `p3 = p1 + p2` ➡ $(25, 43) = (10, 20) + (15, 23)$
2. `p3 = p3 + 6` ➡ $(31, 49) = (25, 43) + (6, 6)$ // 表示 x, y 座標都加 6
3. `++p3` ➡ $(32, 50) = (25, 43) + (1, 1)$ // 表示 x, y 座標都加 1

執行結果：

```
obj1座標->x座標: 10        y座標:20
obj2座標->x座標: 15        y座標:23
obj3座標->x座標: 0         y座標:0

p1+p2=p3
p3座標->x座標: 25          y座標:43

p3+6
p3座標->x座標: 31          y座標:49

++p3
p3座標->x座標: 32          y座標:50

Program finished with exit code 0
Press any key to exit
```

題解說明

請翻閱 Ch15 PPT 、與此 pdf 下方附錄，來實現以下運算子（可點擊下列）：

- Addition `+`
- Assignment `=`
- Post-increment `++`

完整程式碼

```cpp
#include <iostream>
using namespace std;

class Point
{
  private:
    int x;
    int y;

  public:
    Point(int i = 0, int j = 0) : x(i), y(j) {}

    void show()
    {
        cout << "座標->x座標: " << x << "\t y座標:" << y << endl;
```

```
    }


    Point operator+(Point const& b)
    {
        return Point(this->x + b.x, this->y + b.y);
    }

    Point operator+(int const& b)
    {
        return Point(this->x + b, this->y + b);
    }

    Point& operator++()
    {
        this->x += 1;
        this->y += 1;

        return *this;
    }

    Point& operator=(Point const& b)
    {
        this->x = b.x;
        this->y = b.y;

        return *this;
    }
};

int main()
{
    Point p1(10, 20);
    Point p2(15, 23);
    Point p3;

    cout << "obj1"; p1.show();
    cout << "obj2"; p2.show();
    cout << "obj3"; p3.show();


    p3 = p1 + p2;
    cout << "\n" << "p1+p2=p3" << endl;
    cout << "p3"; p3.show();

    p3 = p3 + 6;
    cout << "\n" << "p3+6" << endl;
    cout << "p3"; p3.show();

    ++p3;
    cout << "\n" << "++p3" << endl;
    cout << "p3"; p3.show();

    return 0;
}
```

1A. operator 附錄

多看多背多學習。

<table>
<tr><th>Operators</th><th>Code example for <code>object p1, p2</code></th></tr>
<tr><td rowspan="4">Increment/
Decrement:
<code>++</code> and <code>--</code></td><td>前置運算子 pre-:
<pre>
1 Point p1(1, 1);
2 Point old_p1 = ++p1; // old_p1 is (2, 2), and p1 is now (2, 2)
3
4 Point p2(5, 5);
5 Point old_p2 = --p2; // old_p2 is (4, 4), and p2 is now (4, 4)
</pre></td></tr>
<tr><td><pre>
Point& operator++()
{
    this->x += 1;
    this->y += 1;

    return *this;
}
</pre></td></tr>
<tr><td><pre>
Point& operator--()
{
    this->x -= 1;
    this->y -= 1;

    return *this;
}
</pre></td></tr>
<tr><td>後置運算子 post-:
<pre>
1 Point p1(1, 1);
2 Point old_p1 = p1++; // old_p1 is (1, 1), but p1 is now (2, 2)
3
4 Point p2(5, 5);
5 Point old_p2 = p2--; // old_p2 is (5, 5), but p2 is now (4, 4)
</pre></td></tr>
<tr><td><pre>
Point operator++(int)
{
    Point temp = *this;

    this->x += 1;
    this->y += 1;

    return temp;
}
</pre></td></tr>
<tr><td><pre>
Point operator--(int)
{
    Point temp = *this;
    this->x -= 1;
    this->y -= 1;
    return temp;
}
</pre></td></tr>
<tr><td rowspan="2">Assignment: <code>=</code></td><td>賦值運算子 Assignment Operator:
<pre>
1 Point p1(1, 1);
2 Point p2(5, 5);
3 Point p3(2, 6);
4 p1 = p2 = p3; // Evaluates right-to-left: p2 becomes (2, 6), then p1
  becomes (2, 6)
</pre></td></tr>
<tr><td><pre>
Point& operator=(Point const& b)
{
    this->x = b.x;
    this->y = b.y;

    return *this;
}
</pre></td></tr>
</table>

Addition: `+`

加法運算子 Addition Operator:

```
Point p1(1, 1), p2(2, 3);
Point p3 = p1 + p2; // p3 is (3, 4)
Point p4 = p1 + 5;  // p4 is (6, 6)
```

```
Point operator+(Point const& b)
{
    return Point(
        this->x + b.x,
        this->y + b.y
    );
}
```

```
Point operator+(int const& b)
{
    return Point(
        this->x + b,
        this->y + b
    );
}
```